R1.04 - Architecture et Systèmes d'Exploitation : Gestion des Processus et Signaux POSIX

Thème principal : Systèmes d'exploitation, Processus sous Unix/Linux, Ordonnancement et Signaux POSIX

Synthèse exécutive : Ce cours pose les fondements indispensables de la gestion des tâches sous les systèmes d'exploitation de type Unix/Linux. Il détaille la transition critique entre un programme passif stocké sur disque et un processus dynamique résidant en mémoire vive (RAM). Pour garantir l'exécution simultanée et fluide de multiples applications, le noyau s'appuie sur le multitâche préemptif et un cycle de vie à trois états fondamentaux : Élu, Prêt et Bloqué. La séance aborde également l'arborescence généalogique des processus (PID/PPID) issue du processus racine systemd/init (PID 1), l'utilisation des appels système fork() et exec(), ainsi que la communication asynchrone par signaux POSIX (SIGINT, SIGTERM, SIGKILL, SIGSTOP, SIGCONT).

Du Programme Statique au Processus Dynamique : Cycle de Vie, Attributs et Multitâche

Pour qu'un ordinateur exécute des instructions, le système d'exploitation doit faire basculer le code d'un état statique (sur le disque) à un état dynamique (en mémoire vive). Le noyau gère cette exécution en orchestrent les processus à travers un cycle de vie rigoureux. Lorsqu'un processus est à l'état 'Élu', il utilise activement le processeur. Si son quantum de temps expire, l'ordonnanceur le replace dans la file d'attente à l'état 'Prêt'. S'il effectue une opération d'entrée/sortie (comme lire un fichier sur le disque ou attendre qu'un utilisateur tape du texte), il passe à l'état 'Bloqué' afin de ne pas gaspiller inutilement des cycles CPU. Une fois l'opération terminée, le noyau le bascule d'abord à l'état 'Prêt' et non directement à l'état 'Élu'. Tous les processus descendent d'une structure arborescente stricte dont la racine absolue est le processus systemd ou init (PID 1).

- • Programme vs Processus : Le programme est un fichier exécutable binaire stocké passivement sur le disque dur. Le processus est l'instance active et dynamique de ce programme en mémoire RAM, consommant du temps CPU et des ressources.
- • Multitâche Préemptif : Mécanisme par lequel l'ordonnanceur (scheduler) du noyau alloue et retire dynamiquement de minuscules tranches de temps processeur (quanta) aux processus pour simuler une exécution simultanée.
- • Bloc de Contrôle du Processus (PCB) : Structure interne du noyau regroupant toutes les métadonnées d'un processus : PID, PPID, état actuel, registres du processeur, espace mémoire assigné, UID de l'utilisateur et descripteurs de fichiers ouverts.
- • Filiation PID / PPID : Chaque processus possède un identifiant unique (PID - Process ID) et référence l'identifiant de son créateur (PPID - Parent Process ID). Le processus racine universel est systemd ou init (PID 1).
- • Appels Système fork() et exec() : fork() duplique à l'identique le processus parent pour engendrer un processus enfant, tandis qu'exec() charge un nouveau programme binaire par-dessus le code de l'enfant.
- • Les Trois États du Processus : Élu (exécute des instructions sur le processeur), Prêt (possède ses ressources mais attend que l'ordonnanceur lui attribue le CPU), Bloqué (suspendu en attente d'une E/S disque ou d'un événement externe).

Cas concret enseignant : *L'analogie du livre de cuisine : Le programme stocké sur le disque dur est comparé à un livre de recettes sagement rangé dans une bibliothèque (statique, passif, ne consommant ni électricité ni ingrédients). Le processus est le cuisinier en train d'exécuter activement la recette dans sa cuisine (dynamique, consommant de l'espace sur le plan de travail/RAM, du temps de cuisson/CPU et manipulant des ustensiles/descripteurs de fichiers).*

Contrôle et Communication Inter-Processus : Les Signaux POSIX

Les signaux constituent la forme de communication inter-processus (IPC) la plus élémentaire sous Unix. Lorsqu'un signal est émis vers un processus cible, le noyau interrompt momentanément son fil d'exécution normal pour le forcer à réagir. Bien que le développeur puisse écrire du code pour intercepter (catch) ou ignorer la majorité des

signaux, la norme POSIX impose des garde-fous stricts : les signaux SIGKILL (9) et SIGSTOP (19) ne peuvent jamais être interceptés ou masqués. Ils sont traités directement par le noyau pour donner à l'administrateur le pouvoir absolu de reprendre le contrôle d'une machine face à des processus récalcitrants.

• Signal POSIX : Interruption logicielle asynchrone transmise par le noyau à un processus pour l'informer qu'un événement s'est produit ou lui ordonner d'exécuter une action spécifique.

• Gestionnaire de signal (Signal Handler) : Fonction logicielle qu'un programme peut enregistrer pour intercepter (catch) un signal et en personnaliser le comportement au lieu d'exécuter l'action par défaut.

• SIGINT (Signal 2) : Déclenché par la combinaison de touches 'Ctrl + C' dans le terminal ; demande l'interruption modérée d'une tâche interactive.

• SIGTERM (Signal 15) : Signal de terminaison par défaut envoyé par la commande 'kill'. Il invite le processus à libérer proprement ses ressources et sauvegarder ses données avant de s'arrêter.

• SIGKILL (Signal 9) : Ordre d'extinction inconditionnel géré directement par le noyau Linux. Il détruit immédiatement le processus sans sommation et sans possibilité d'être intercepté ou ignoré.

• Job Control (SIGSTOP 19 / SIGCONT 18) : La combinaison 'Ctrl + Z' émet SIGSTOP pour suspendre un processus en arrière-plan ; les commandes 'fg' ou 'bg' émettent SIGCONT pour reprendre son exécution.

Cas concret enseignant : *L'analogie de l'expulsion de l'immeuble : SIGTERM (15) est le gardien d'immeuble qui frappe poliment à votre porte pour vous demander de quitter les lieux (vous disposez du temps nécessaire pour ranger vos affaires, sauvegarder vos fichiers et fermer la porte à clé). SIGKILL (9) est une intervention d'urgence des forces de sécurité qui défonce la porte et vous expulse sur-le-champ sans vous laisser la moindre seconde pour toucher à quoi que ce soit.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi la commande 'kill 9 1234' renvoie-t-elle une erreur dans le terminal alors que 'kill -9 1234' fonctionne parfaitement ?
Réponse du professeur : Parce que sans le tiret '-', le shell interprète le premier argument '9' comme étant le PID du processus cible et non comme le numéro du signal. La commande cherche donc à envoyer le signal par défaut (SIGTERM, 15) à un processus dont le PID est 9, ce qui échoue ou cible un autre processus système. Le tiret '-9' est indispensable pour indiquer explicitement qu'il s'agit d'une option spécifiant le numéro du signal.

Question étudiant : Qu'est-ce qu'un processus dit 'Zombie' et comment peut-on le détruire à l'aide de la commande kill ?
Réponse du professeur : Un processus zombie est un processus qui a terminé son exécution mais dont l'entrée dans la table des processus (le PCB) subsiste car le processus parent n'a pas encore lu son code de retour (via l'appel système wait()). Précision fondamentale : un processus zombie est déjà mort, il ne consomme plus de CPU ni de mémoire RAM, juste une ligne dans la table des noyaux. Par conséquent, **on ne peut pas le tuer avec la commande kill** (puisqu'il est déjà mort). La seule solution pour le faire disparaître est de tuer son processus parent afin que le sous-système init récupère et nettoie l'orphelin.

■ Exercice & Démonstration d'Application

Énoncé : Dans un terminal Bash sous Linux, un étudiant exécute la séquence de commandes suivante : 1. sleep 600 & 2. ps -ef | grep sleep 3. kill -15 Explicitez le rôle du symbole '&', détaillez le traitement réalisé lors de l'envoi du signal -15 et décrivez le résultat de la vérification ultérieure par la commande ps.

Résolution pas à pas :
Étape 1 : Le symbole '&' placé à la fin de la première ligne indique au shell d'exécuter la commande 'sleep 600' en arrière-plan (background). Le shell affiche immédiatement l'identifiant de tâche (job ID) ainsi que le PID attribué par le système (ex: [1] 4521) et redonne instantanément la main à l'utilisateur pour lui permettre de taper de nouvelles commandes. Étape 2 : La commande 'ps -ef | grep sleep' interroge l'ensemble des processus du système d'exploitation et filtre les résultats pour afficher uniquement la ligne correspondant à notre commande 'sleep'. Cela permet de relever avec certitude son PID exact (4521) ainsi que son PPID (qui correspond au PID du shell Bash interactif). Étape 3 : La commande 'kill -15 4521' transmet le signal SIGTERM (Signal 15) au processus identifié. L'utilitaire 'sleep' ne disposant pas d'un gestionnaire personnalisé pour intercepter ce signal, il applique le

comportement par défaut défini par le système d'exploitation : il libère ses ressources et s'arrête proprement.

Résultat : Le processus 'sleep 600' est définitivement arrêté de manière propre. Une nouvelle exécution de la commande 'ps -ef | grep sleep' confirme sa disparition totale de la table des processus actifs du système.

■ Auto-Évaluation Flash d'Examen

Q : Quel est le nom et le PID du tout premier processus utilisateur démarré par le noyau Linux lors du démarrage de la machine ?

R : Il s'agit du processus systemd (ou init), qui possède systématiquement le PID 1.

Q : Quel signal POSIX est généré et transmis au processus s'exécutant au premier plan lorsqu'un utilisateur presse la combinaison de touches 'Ctrl + C' dans son terminal ?

R : Le signal SIGINT (Signal 2), qui demande l'interruption douce de la tâche interactive.

Q : Pourquoi le signal SIGKILL (Signal 9) ne peut-il jamais être intercepté, masqué ou ignoré par un programme informatique ?

R : Parce qu'il est traité directement et exclusivement par le noyau du système d'exploitation, garantissant à l'administrateur une porte de sortie inconditionnelle face à un processus bloqué ou malveillant.